Angular Velocity Calibration

Angular Velocity Calibration

1. Course Content
2. Preparation
3. Calibration Demonstration
4. Source Code Analysis

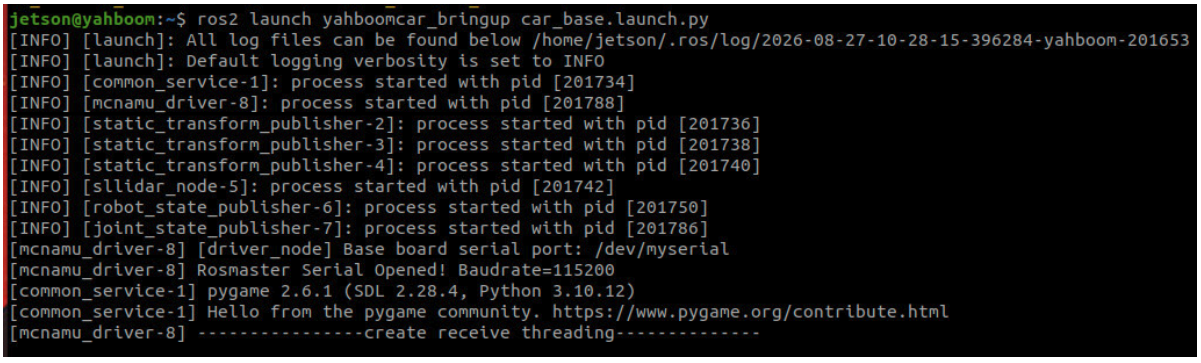
1. Course Content

- Verify the rotation accuracy of the chassis wheel odometry (computed from the motor encoder data) and carry out angular velocity calibration

> **Important**
>
> The car has already been calibrated at the factory, so there is no need to calibrate it on first use. If you find the odometry rotation error to be relatively large during later use, refer to this tutorial to calibrate it

2. Preparation

```
ros2 launch yahboomcar_bringup car_base.launch.py
```



3. Calibration Demonstration

- Run the angular velocity calibration program

```
ros2 run common_demo calibrate_angular_z
```

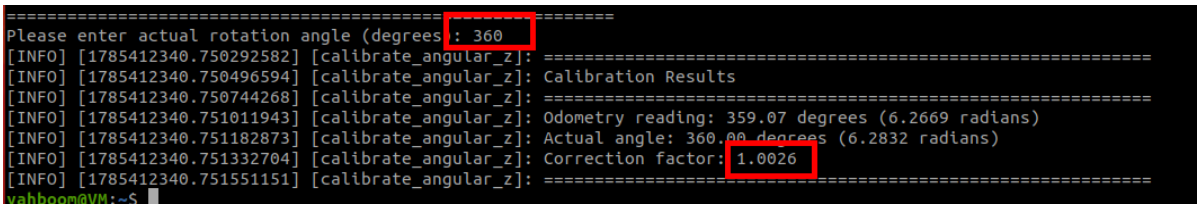
> **Tip**
>
> - Optional extra startup parameters

Parameter Name	Type	Default Value	Description
test_angle_deg	float	360.0	Test rotation angle (degree)
angular_speed	float	0.35	Rotation speed (rad/s)
tolerance_deg	float	0.02	Allowed error (degree)

- Example of the parameter usage format

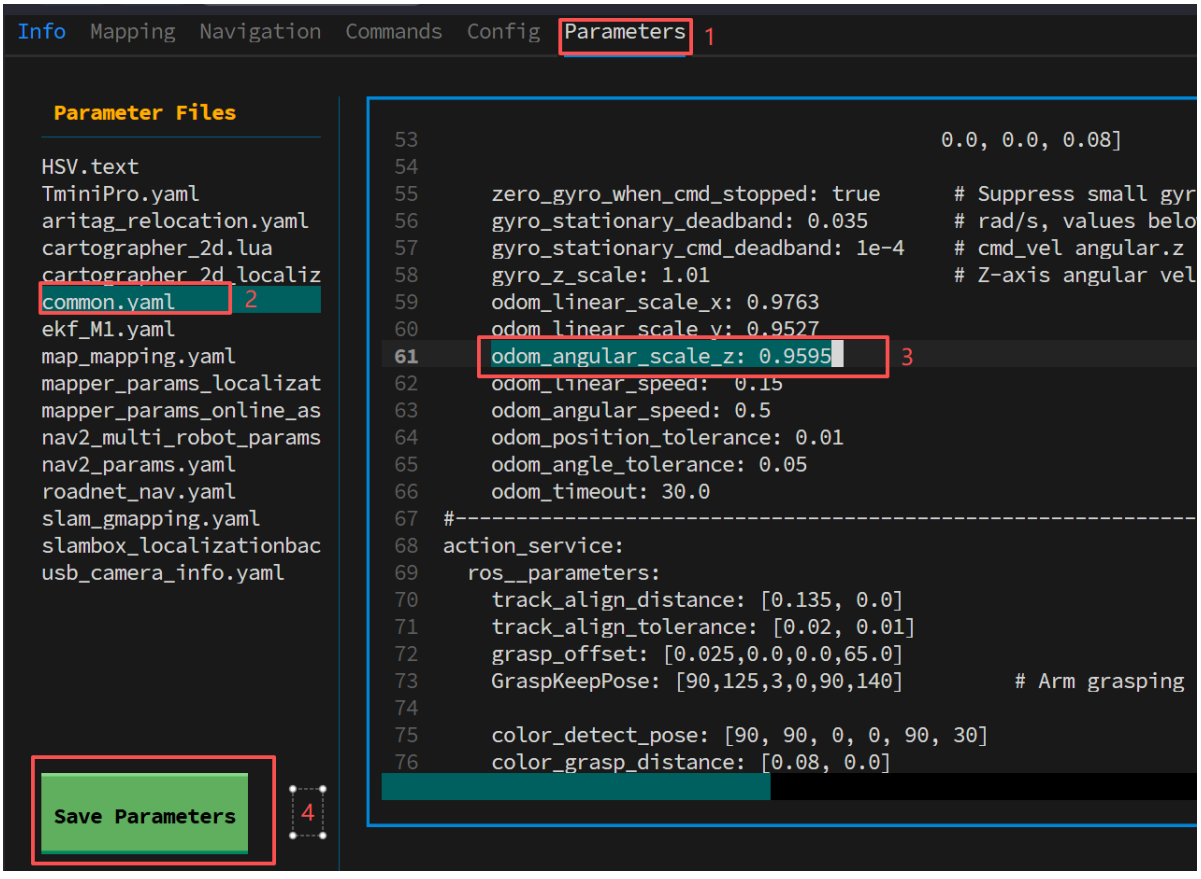
```
ros2 run common_demo calibrate_angular_z --ros-args -p test_angle_deg:=360.0
-p angular_speed:=0.35
```

- The default test angle is 360 degrees. After the car stops rotating, measure the actual rotation angle and enter the measured actual rotation angle into the terminal; the terminal will automatically calculate the correction coefficient. Enter the chassis's actual rotation angle



- Apply the calibration coefficient

```
robot_manage gui
```



4. Source Code Analysis

- Source code path

```
$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/common_demo/common_demo/calibration/ca
librate_angular_z.py
```

1. `odom_callback` — **odometry yaw angle callback**

- Extract the quaternions `qz` and `qw` from the odometry message, and convert them into the yaw angle `current_yaw` through `2.0 * atan2(qz, qw)`
- On the first reception, record the initial yaw angle `start_yaw` and set the `odom_received` flag

2. `control_loop` — **main control loop of the angular velocity calibration**

- Calculate the yaw increment `delta_yaw` between two adjacent callbacks, normalized by ±π to avoid angle jumps
- Accumulate the yaw increments to obtain the total rotation angle `accumulated_angle`
- When the accumulated angle reaches the target angle `test_angle`, stop rotating, output the odometry reading, and set `waiting_for_input` to wait for the user to enter the actual rotation angle
- When the target has not been reached, decelerate in stages according to the remaining angle (< 0.1 rad minimum speed, 0.1~0.3 rad linear deceleration, > 0.3 rad full speed)
- Publish a `Twist` angular velocity command to control the car's counter-clockwise rotation

```
    def odom_callback(self, msg):
        """Odometry data callback - extract yaw from quaternion"""
        # Convert quaternion to yaw angle
        qz = msg.pose.pose.orientation.z
        qw = msg.pose.pose.orientation.w
        self.current_yaw = 2.0 * atan2(qz, qw)

        if not self.odom_received:
            self.odom_received = True
            self.start_yaw = self.current_yaw
            self.previous_yaw = self.current_yaw
            self.get_logger().info(f'Odometry data received, start yaw:
{self.start_yaw:.4f} radians')

    def control_loop(self):
        """Main control loop for angular calibration"""
        if not self.odom_received:
            return

        if not self.test_started:
            self.start_yaw = self.current_yaw
            self.previous_yaw = self.current_yaw
            self.accumulated_angle = 0.0
            self.test_started = True

        delta_yaw = self.current_yaw - self.previous_yaw

        while delta_yaw > 3.14159:
            delta_yaw -= 2.0 * 3.14159
        while delta_yaw < -3.14159:
            delta_yaw += 2.0 * 3.14159

        # Accumulate the angle (always positive for counter-clockwise rotation)
        self.accumulated_angle += abs(delta_yaw)
```

```
        # Update previous yaw
        self.previous_yaw = self.current_yaw

        # Use accumulated angle for calibration
        rotated_angle = self.accumulated_angle

        # Calculate error
        error = self.test_angle - rotated_angle

        # Check if target reached
        if abs(error) < self.tolerance and not self.test_completed:
            self.test_completed = True
            self.get_logger().info('='*60)
            self.get_logger().info('Calibration test rotation completed!')
            self.get_logger().info('='*60)
            # Stop robot
            self.cmd_vel_pub.publish(Twist())
            self.timer.cancel()
            # Display odometry reading
            self.odom_angle = rotated_angle
            odom_angle_deg = self.odom_angle * RAD_TO_DEG
            self.get_logger().info(f'Odometry reading: {odom_angle_deg:.2f}
degrees ({self.odom_angle:.4f} radians)')
            self.get_logger().info('='*60)
            self.get_logger().info('Enter actual rotation angle...')

            # Set flag to notify main function to wait for user input
            self.waiting_for_input = True
            return

        # Target not reached, continue rotating
        move_cmd = Twist()
        angle_to_go = abs(error)

        # Angular velocity ramping: reduce speed when approaching target to
prevent overshoot
        min_speed = max(0.03, self.angular_speed * 0.2)
        close_threshold = 0.1    # Below this (~5.73°): minimum speed
        ramp_threshold = 0.3     # Below this (~17.19°): linear deceleration

        if angle_to_go < close_threshold:
            ramp_speed = min_speed
        elif angle_to_go < ramp_threshold:
            ramp_speed = min_speed + (self.angular_speed - min_speed) * (
                (angle_to_go - close_threshold) / (ramp_threshold -
close_threshold))
        else:
            ramp_speed = self.angular_speed

        # Rotate counter-clockwise (positive angular.z)
        move_cmd.angular.z = ramp_speed
        self.cmd_vel_pub.publish(move_cmd)
```